Name: Stephen Wairua

A) User Manual Procedure

Creating and Activating a Python Virtual Environment and Installing a Package

Creating and Activating a Python Virtual Environment and Installing a Package

Prerequisites
Before starting, make sure you have:

1. A computer with Python 3 installed.
2. A terminal or command prompt.
3. Basic knowledge of opening and using a terminal.
4. An internet connection for downloading the package.
5. Permission to create files and folders on your computer.
6. A project folder where you want to create the virtual environment.

Procedure
Step 1: Open a terminal
Open Command Prompt, PowerShell, or a terminal application.

Expected result: A terminal window opens and displays a command prompt.

Step 2: Create a project folder
Enter the following command:

mkdir my_python_project

Expected result: A folder named my_python_project is created in the current location.

Step 3: Enter the project folder

Run:

cd my_python_project

Expected result: The terminal is now working inside the my_python_project folder.

Step 4: Create a virtual environment
Run:
python -m venv .venv

Expected result: A .venv folder is created inside the project folder.

Note: On some systems, you may need to use python3 instead of python.

Step 5: Activate the virtual environment on Windows
If you are using Windows PowerShell, run:

.venv\Scripts\Activate.ps1

Expected result: The terminal prompt displays (.venv) at the beginning.

For Windows Command Prompt, use:

.venv\Scripts\activate

Step 6: Activate the virtual environment on macOS or Linux
If you are using macOS or Linux, run:

source .venv/bin/activate

Expected result: The terminal prompt displays (.venv) at the beginning.

Step 7: Install the requests package
Run:

python -m pip install requests

Expected result: The terminal downloads and installs the requests package and reports a successful installation.

Step 8: Verify the package installation
Run:

python -m pip show requests

Expected result: The terminal displays information about the installed requests package, including its version and installation location.

Step 9: Deactivate the virtual environment
Run:

deactivate

Expected result: The (.venv) text disappears from the terminal prompt.

Screenshot Description
Include a screenshot showing the terminal immediately after the package installation has completed successfully.

The screenshot should show:

The active (.venv) environment in the terminal prompt.

The python -m pip install requests command.

A successful installation message.

The command prompt ready for the next command.

The screenshot should not contain personal information such as usernames, passwords, API keys, or private file paths.

Troubleshooting
Error: 'python' is not recognized or python: command not found
This usually means Python is either not installed or the Python command is not available in the system's PATH.

Solution: Confirm that Python is installed by running:

python --version

If that command fails, try:

python3 --version

If python3 works, use python3 instead of python in the remaining commands. If neither command works, install Python and ensure that it is added to the system PATH during installation.

Completion Check
The procedure is complete when:

The .venv folder exists inside my_python_project.

The terminal shows (.venv) when the environment is active.

The requests package is installed successfully.

python -m pip show requests displays the package information.

B) API Reference Entry

Create a Task

Method: POST
Endpoint: `/api/v1/projects/{project_id}/tasks`

Description

The authenticated user must have permission to create tasks in the project.

The endpoint returns the newly created task, including its assigned user, due date, priority, and creation timestamp.

Path Parameters

Parameter	Type	Required	Description
`project_id`	Integer	Yes	The unique ID of the project where the task will be created.

Query Parameters

This endpoint does not require any query parameters.

Request Headers

Header	Required	Description
`Authorization`	Yes	Bearer token used to authenticate the user.
`Content-Type`	Yes	Must be set to `application/json`.
`Accept`	Yes	Must be set to `application/json` to request a JSON response.

Authentication

The endpoint uses Bearer token authentication.

```
Authorization: Bearer <access_token>
```

The access token must belong to an authenticated user who has permission to create tasks in the specified project.

Request Body

The request body must be a JSON object containing the following fields:

Field	Type	Required	Description
`title`	String	Yes	The title or short name of the task.
`description`	String	No	Additional information about the task.
`assignee_id`	Integer	Yes	The unique ID of the user assigned to the task.
`due_date`	String	Yes	The date by which the task should be completed. Use the `YYYY-MM-DD` format.

`priority`	String	Yes	The task priority. Accepted values are `low`, `medium`, or `high`.

Example Request

```
POST /api/v1/projects/125/tasks HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json
Accept: application/json

{
  "title": "Prepare project progress report",
  "description": "Compile the team's progress and outstanding
tasks for the monthly review.",
  "assignee_id": 42,
  "due_date": "2026-11-15",
  "priority": "high"
}
```

Response Codes

HTTP Status	Meaning	When It Occurs
`201 Created`	Task created successfully	The request is valid and the new task has been created.
`400 Bad Request`	Invalid request	The request body is malformed or contains invalid data.
`401 Unauthorized`	Authentication required or failed	The access token is missing, invalid, or expired.
`403 Forbidden`	Permission denied	The authenticated user does not have permission to create tasks in the project.
`404 Not Found`	Project or user not found	The specified `project_id` or `assignee_id` does not exist or cannot be accessed.

`409 Conflict`	Request conflicts with current data	The task cannot be created because it conflicts with an existing resource or project state.
`422 Unprocessable Entity`	Validation failed	The request is syntactically valid, but one or more fields fail validation.
`500 Internal Server Error`	Server error	An unexpected error occurs while processing the request.
`503 Service Unavailable`	Service unavailable	The API is temporarily unable to process the request, such as during maintenance or an outage.

Successful Response

Status: `201 Created`

```
{
  "id": 9876,
  "project_id": 125,
  "title": "Prepare project progress report",
  "description": "Compile the team's progress and outstanding
tasks for the monthly review.",
  "assignee": {
    "id": 42,
    "name": "Amina Hassan"
  },
  "due_date": "2026-11-15",
  "priority": "high",
  "status": "pending",
  "created_by": {
    "id": 18,
    "name": "David Mwangi"
  },
  "created_at": "2026-10-05T14:32:18Z"
}
```

Response Field Descriptions

Field	Type	Description
`id`	Integer	Unique identifier of the newly created task.
`project_id`	Integer	ID of the project containing the task.
`title`	String	Title of the task.
`description`	String	Description provided when the task was created.
`assignee`	Object	Information about the user assigned to the task.
`assignee.id`	Integer	Unique ID of the assigned user.
`assignee.name`	String	Display name of the assigned user.
`due_date`	String	Task deadline in `YYYY-MM-DD` format.
`priority`	String	Task priority: `low`, `medium`, or `high`.
`status`	String	Current task status. A newly created task starts as `pending`.
`created_by`	Object	User who created the task.
`created_at`	String	Date and time when the task was created, in ISO 8601 UTC format.

Validation Rules

- `title` must be provided and must not be empty.

- `assignee_id` must identify an existing user who can be assigned to the project.
- `due_date` must use the `YYYY-MM-DD` format.
- `priority` must be one of `low`, `medium`, or `high`.
- `description` is optional.
- The authenticated user must have permission to create tasks in the specified project.